

Goroutines - Cheat Sheet

Contents

1. [Starting a goroutine](#)
2. [Goroutines vs OS threads](#)
3. [The M:N scheduler](#)
4. [GOMAXPROCS](#)
5. [Stopping a goroutine](#)
6. [Communicating with goroutines](#)
7. [sync primitives](#)
8. [The race detector](#)
9. [Common patterns](#)
10. [Best practices](#)
11. [Common gotchas](#)
12. [Quick reference](#)

A goroutine is a function running concurrently with other code, managed by the Go runtime instead of the OS. The `go` keyword starts one. The runtime multiplexes thousands or millions of goroutines onto a small pool of OS threads.

Cheap to create (8 KB initial stack, grows as needed), cheap to schedule (cooperative + preemptive since Go 1.14), and the only way to do concurrency in Go.

1. Starting a goroutine

```
go doWork()           // call doWork() in a new goroutine
go func() {           // or an anonymous function
    fmt.Println("hi")
}()

// With captured variables
name := "alice"
go func() {
    fmt.Println(name) // closes over name
}()
```

The `go` statement returns immediately. The caller doesn't wait. The result of the function is discarded.

A goroutine ends when its function returns. There's no way to forcibly kill one from outside. Shutdown is cooperative: context, channels, or shared flags.

2. Goroutines vs OS threads

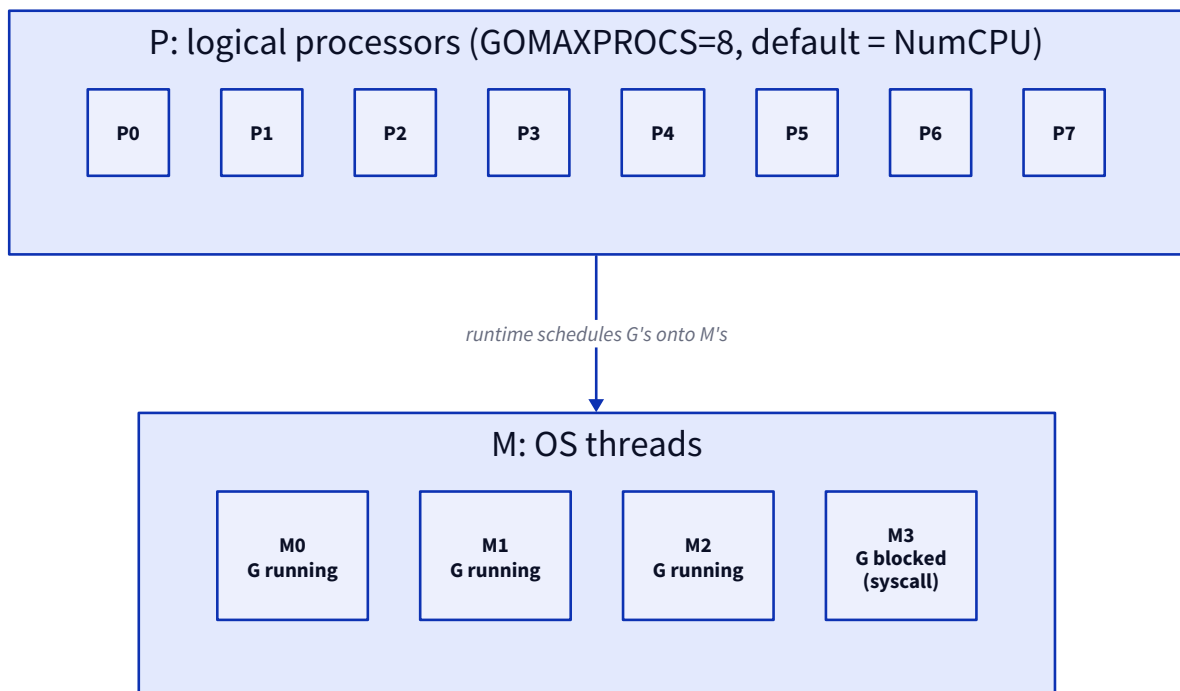
Aspect	OS thread	Goroutine
Created by	Kernel	Go runtime
Initial stack	1-8 MB	~8 KB, grows on demand
Switching cost	Microseconds (syscall)	Hundreds of nanoseconds
How many fit in RAM	Thousands	Millions
Scheduling	OS scheduler	Go's M:N scheduler
Identity	Has a thread ID	No usable ID (intentional)

One Go process can run a million goroutines on 8 OS threads. That's the whole point.

3. The M:N scheduler

Go's runtime maps M goroutines onto N OS threads. Three components:

- **G** (goroutine): the work to do.
- **M** (machine): an OS thread.
- **P** (processor): a logical processor that holds a queue of runnable G's. `GOMAXPROCS` sets the count.



Each P has a local run queue. An idle P can steal goroutines from a busy P's queue (work-stealing). A goroutine blocking in a syscall hands its P off to another M. Network I/O uses the netpoller, which doesn't tie up a thread.

Since Go 1.14, the scheduler is preemptive. A goroutine can be paused mid-function (every 10ms by default). Before 1.14, a tight loop without a function call could starve the scheduler.

4. GOMAXPROCS

The number of OS threads that can run Go code simultaneously. Defaults to the number of logical CPUs.

```
import "runtime"

runtime.GOMAXPROCS(4)           // pin to 4
n := runtime.GOMAXPROCS(0)      // read current
```

In containers, `GOMAXPROCS` historically defaulted to the host CPU count, which is wrong when CPU limits are set. Use `automaxprocs` (Uber library) or set `GOMAXPROCS` explicitly from the cgroup limit. Modern Go versions read the cgroup limit automatically when set.

5. Stopping a goroutine

A goroutine stops when its function returns. You can't kill one externally. Three idiomatic ways to coordinate shutdown.

Channel close

```
done := make(chan struct{})

go func() {
    for {
        select {
        case <-done:
            return
        case <-time.After(1 * time.Second):
            doWork()
        }
    }
}()

// Later: signal stop
close(done)
```

`<-done` unblocks when the channel is closed. Closing a channel broadcasts to every receiver.

context.Context

The standard way for anything that involves a request lifecycle, timeout, or cancellation.

```
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

go func() {
    for {
        select {
        case <-ctx.Done():
            return
        case <-time.After(time.Second):
            doWork()
        }
    }
}()

// Later
cancel()
```

`ctx.Done()` returns a channel that closes when the context is canceled or times out. Pass `ctx` down through every function that might need to bail out.

Shared atomic flag

For tight loops where channel ops would be overhead.

```
var stop atomic.Bool

go func() {
    for !stop.Load() {
        doWork()
    }
}()

stop.Store(true)
```

Use sparingly. Most cases want a context or a channel.

6. Communicating with goroutines

Two patterns dominate.

Channels (CSP-style)

Send and receive values to coordinate.

```
ch := make(chan int)

go func() {
    ch <- 42
}()

v := <-ch    // receives 42
```

Buffered:

```
ch := make(chan int, 100)    // 100-slot buffer
```

Channels get full coverage in their own sheet. Quick rules:

- Send on nil channel blocks forever.
- Receive on nil channel blocks forever.
- Send on closed channel panics.
- Receive on closed channel returns the zero value immediately (use the two-value form to detect).

Shared memory with mutexes

When channels are awkward, use `sync.Mutex` to protect shared state.

```
type Counter struct {
    mu sync.Mutex
    n int
}

func (c *Counter) Inc() {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.n++
}

func (c *Counter) Value() int {
    c.mu.Lock()
    defer c.mu.Unlock()
    return c.n
}
```

`sync.RWMutex` for read-heavy workloads. Many readers, one writer.

The Go proverb says “share memory by communicating, don’t communicate by sharing memory”. In practice both work. Channels are good for coordination and data flow. Mutexes are good for protecting state.

7. sync primitives

Primitive	Use for
<code>sync.Mutex</code>	Mutual exclusion
<code>sync.RWMutex</code>	Read-write lock
<code>sync.WaitGroup</code>	Wait for N goroutines to finish
<code>sync.Once</code>	Run something exactly once
<code>sync.Map</code>	Concurrent map (specific use cases)
<code>sync.Pool</code>	Reusable temporary objects
<code>sync.Cond</code>	Condition variable (rarely used in Go)
<code>atomic.Int64</code> , <code>atomic.Bool</code> , etc.	Lock-free atomic ops

WaitGroup

Wait for a fixed set of goroutines to finish.

```
var wg sync.WaitGroup

for i := 0; i < 10; i++ {
    wg.Add(1)
    go func(id int) {
        defer wg.Done()
        process(id)
    }(i)
}

wg.Wait()
```

Always call `wg.Add(n)` before starting the goroutines. Calling `wg.Add` inside the new goroutine races with `Wait()`. The `go vet` tool catches this.

Once

Run an initializer exactly once across all goroutines.

```

var (
    once sync.Once
    cfg *Config
)

func GetConfig() *Config {
    once.Do(func() {
        cfg = loadConfig()
    })
    return cfg
}

```

Idiomatic for lazy singletons.

sync.Map

Concurrent map. Two specific cases where it beats `map` + `Mutex`:

- Many goroutines reading mostly-stable keys.
- Many goroutines writing disjoint key sets.

For the common case of “I have a map and need to protect it”, plain map with a mutex is faster.

8. The race detector

Built into the Go toolchain. Pass `-race` when building or testing.

```

go test -race ./...
go build -race
go run -race main.go

```

It instruments memory accesses and reports any unsynchronized read/write of the same memory from different goroutines. 5-10x slower at runtime and 5-10x more memory, so only for dev and tests.

A passing race test doesn’t prove correctness. It proves the test didn’t observe a race. Keep `-race` in CI.

9. Common patterns

Worker pool

```
func main() {
    jobs := make(chan int, 100)
    results := make(chan int, 100)

    // 5 workers
    for w := 1; w <= 5; w++ {
        go worker(w, jobs, results)
    }

    // Send 9 jobs
    for j := 1; j <= 9; j++ {
        jobs <- j
    }
    close(jobs)

    // Collect 9 results
    for r := 1; r <= 9; r++ {
        fmt.Println(<-results)
    }
}

func worker(id int, jobs <-chan int, results chan<- int) {
    for j := range jobs {
        time.Sleep(time.Second)
        results <- j * 2
    }
}
```

Fan-out, fan-in

Many goroutines producing into one channel, one goroutine consuming.

```

func fanIn(channels ...<-chan int) <-chan int {
    out := make(chan int)
    var wg sync.WaitGroup
    wg.Add(len(channels))

    for _, c := range channels {
        go func(ch <-chan int) {
            defer wg.Done()
            for v := range ch {
                out <- v
            }
        }(c)
    }

    go func() {
        wg.Wait()
        close(out)
    }()

    return out
}

```

Pipeline

Stages connected by channels. Each stage is one or more goroutines.

```

func generate(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for _, n := range nums {
            out <- n
        }
    }()
    return out
}

func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for n := range in {
            out <- n * n
        }
    }()
    return out
}

// Usage
for v := range square(generate(1, 2, 3, 4)) {
    fmt.Println(v)
}

```

errgroup

The `golang.org/x/sync/errgroup` package runs a set of goroutines, returns the first error, and cancels the rest via context.

```
import "golang.org/x/sync/errgroup"

g, ctx := errgroup.WithContext(context.Background())

for _, url := range urls {
    url := url
    g.Go(func() error {
        return fetch(ctx, url)
    })
}

if err := g.Wait(); err != nil {
    log.Fatal(err)
}
```

`g.Wait()` returns the first non-nil error from any goroutine. Other goroutines see `ctx.Done()` close and should exit.

Go 1.20+ added `SetLimit(n)` to cap concurrency:

```
g.SetLimit(10)
```

10. Best practices

- **Use goroutines for I/O concurrency.** They're cheap. A million live goroutines doing HTTP requests is fine.
- **Pass a `context.Context` everywhere.** First parameter, named `ctx`. Goroutines listen on `ctx.Done()` for cancellation.
- **Every `go` statement needs a clear shutdown path.** Don't start a goroutine you can't stop.
- **`defer wg.Done()` at the top of the goroutine body.** Pairs with `wg.Add(1)` before `go`.
- **Use `errgroup`** for fan-out work where any error should cancel siblings.
- **Use the race detector in CI.** Catches the bugs you don't.
- **Avoid `recover()` in every goroutine "just in case".** A panic in an unexpected goroutine is a real bug. Let it crash, then fix it. Use `recover` only at well-defined boundaries (HTTP server, message handler).
- **Cap concurrency for unbounded work.** Spawning a goroutine per item from an unbounded source crushes memory. Worker pool or `errgroup.SetLimit`.
- **Don't pass primitive locks by value.** `sync.Mutex` is copyable but the copy is a different lock. `go vet` catches this.
- **Hold the lock for the shortest time possible.** Get the data, release, work on the copy.

11. Common gotchas

- **Loop variable capture (before Go 1.22).** `for i := range items { go func() { use(i) }() }` shares one `i` across all goroutines. Pre-1.22: pass `i` as an argument. Post-1.22: each iteration has its own `i` and this works correctly.
- **Goroutine leaks.** A goroutine blocked on a channel that nobody will ever send to lives forever. Memory and resources held until the process dies. The most common Go bug.
- **`time.After` in a select loop.** Creates a new timer every iteration. Use `time.NewTimer` and reset it, or use `context.WithTimeout`.
- **Forgetting to `cancel()` a context.** Even after the work finished. Always `defer cancel()` right after `WithCancel` / `WithTimeout`.
- **Closing a channel from the receive side.** Closing belongs to the sender. Receivers that close cause senders to panic.
- **Sending on a nil channel.** Blocks forever silently. Check that channels are initialized.
- **`sync.Mutex` zero value works but the struct must not be copied.** Pass the struct containing it by pointer.
- **Tight loop with no scheduling points (pre-Go 1.14).** Could starve other goroutines. Modern Go preempts every ~10ms. Still avoid pointlessly tight loops.
- **Misusing `sync.WaitGroup` reuse.** A `WaitGroup` whose counter went to zero can be reused for the next batch, but only after `Wait()` returns. Mixing `Add` and `Wait` calls across phases causes panics.
- **Logging or panicking without trace context.** A panic stack trace from goroutine 47 of 10,000 isn't useful without a label or request ID.

12. Quick reference

Concept	Key fact
<code>go f()</code>	Run <code>f()</code> in a new goroutine
<code>GOMAXPROCS</code>	Number of OS threads running Go code simultaneously
<code>runtime.Gosched()</code>	Yield to the scheduler manually (rarely needed)
<code>context.Context</code>	The standard cancellation mechanism
<code>sync.WaitGroup</code>	Wait for N goroutines to finish
<code>sync.Mutex</code>	Mutual exclusion
<code>sync.Once</code>	Run something exactly once
<code>sync.Map</code>	Concurrent map for specific access patterns
<code>errgroup.Group</code>	Goroutines with first-error and context cancellation
<code>-race</code> flag	Detect data races at runtime
Goroutine leak	Goroutine blocked on something that never happens
Worker pool	Fixed N goroutines pulling from a job channel

Concept	Key fact
Fan-out / fan-in	Many producers, one consumer
Pipeline	Stages connected by channels